Lab 3 - Architecture Pattern Selection

Problem Statement 44: Database Query Performance Profiler

Name: Vishal Prasath

SRN: PES1UG24CS536

Architecture Selection

I chose Microservices Architecture for the Database Query Performance Profiler system.

Before choosing, I looked at all three styles. Layered architecture is easy to build but the whole application has to be scaled together, and our log ingestion part needs a lot more capacity than the rest. Client-Server puts everything on one server, so if that server goes down we stop collecting logs and lose entries. Microservices lets us split the system into small services that each do one job, which suits our problem better, even though it is a bit more work to deploy and manage.

Reason 1: Different parts of the system have very different load

The NFR says the system must handle up to 5000 slow query log entries per minute without dropping any. Only the Log Ingestion Service gets this constant load. The Query Plan Analyzer and Index Advisor run when there is something to analyse, and the Report Generator only runs once a week for the digest. Since they are separate services, we can run more copies of the ingestion service alone instead of scaling the whole system.

Reason 2: One service failing does not stop the others

Parsing EXPLAIN plans from both PostgreSQL and MySQL is the part most likely to fail on unusual queries. Ingestion sends work to the analyzer through a queue (IPlanAnalysis), so if the analyzer crashes, ingestion keeps saving raw logs to the Metrics Repository and the analyzer picks up the pending jobs when it comes back. We can also update or add a new parser without redeploying everything.

Security Advantage

Only two services (Log Ingestion and Query Plan Analyzer) connect to the production database, and both use read-only access. The DBA and Backend Lead never talk to the services directly - every request from the dashboard goes through the API Gateway, which checks login and user role first. So even if one service is compromised, the attacker does not get access to the other services or their credentials.

Performance Benefit

Ingestion does not wait for analysis to finish. It reads the log, saves the entries in batches and puts the analysis job on the queue, then moves on. This way the system can handle bursts of slow queries without slowing down, and when the queue gets long we can just add more analyzer instances.